

HAL

BOOK II — ARCHITECTURE SPECIFICATION

v0.3 • Chapters 1–3 complete

Book II Charter

Book I defines what must remain true. Book II defines the technical architecture that fulfills those constitutional requirements. Book II is governed by Book I; no architectural decision may weaken or contradict a constitutional rule.

Current Contents

Chapter	Status	Scope
1 — Overall System Architecture	Complete	System planes, authority boundaries, state ownership, trust boundaries, and end-to-end flow.
2 — Runtime Model	Complete	Bootstrap, lifecycle, modes, supervision, degradation, shutdown, and recovery.
3 — Constitutional Kernel	Complete	Protected root state, contracts, authorization ceremony enforcement, protected commit, and recovery authority.
4 — Identity and Continuity	Next	Universal identity, session authentication, delegation, health, and identity lifecycle.

Chapter 1 — Overall System Architecture

Field	Value
Status	Architect draft complete; constitutional and engineering review complete
Version	0.1
Scope	System topology, authority boundaries, major planes, data ownership, and end-to-end flow
Constitutional basis	Decisions 1–58, with primary implementation responsibility for 1–7, 10–16, 24–25, 29, 32, 35–42, 45, 47, 49–51
Owner review	None required at this stage

1. Purpose

This chapter defines the whole HAL system before later chapters specify individual services. It establishes which components are authoritative, how data and actions cross boundaries, and how the system preserves constitutional identity while executing work through replaceable models, services, nodes, and external partners.

2. Architectural Thesis

HAL is a constitutionally governed control system. The constitutional kernel makes and records authoritative decisions; governed services reason, plan, store, observe, and coordinate; capability providers perform bounded work; Presences expose the system to people and the physical world. No provider, node, or interface gains authority merely by participating.

3. System Planes

Plane	Responsibilities	Authority posture
Constitutional plane	Owner identity, policy, authority, authorization ceremony, audit integrity, constitutional state.	Authoritative; only governed mutation paths.
Cognitive-control plane	Intent, planning, evidence synthesis, trust, attention, judgment, scheduling, verification.	Makes or recommends decisions under policy.
State and knowledge plane	Experience Ledger, knowledge graph, decision records, configuration, transactions, outcomes.	Each durable domain has one authoritative owner.
Execution plane	Execution Orchestrator, capability contracts, providers, node runtimes, adapters, transactions.	Bounded execution; cannot alter constitutional state directly.

Plane	Responsibilities	Authority posture
Presence and federation plane	Desktop, phone, voice, watch, robot, APIs, external trust domains, Treaties.	Contextual interfaces; cross-domain exchange passes the Constitutional Firewall.

4. Reference Topology

Owner & Presences

↓ authenticated, authorized requests

HAL Core — Constitutional Kernel + Governed Services

- └ Constitutional plane: identity • policy • authority • audit
- └ Cognitive-control plane: intent • planning • evidence • trust • judgment • scheduling
- └ State plane: ledger • knowledge • transactions • configuration • observability
- └ Execution Orchestrator
 - ↓ capability contract + delegated authority

HAL Node Runtimes & Governed Providers

↓ bounded results, evidence, health, and transaction updates

External Trust Domains / Physical & Digital Environment

5. Constitutional Kernel

The kernel is deliberately small. It contains only what must remain trustworthy when every optional service fails: root identity, authorization, policy issuance and evaluation authority, protected audit rules, transaction commit coordination, cluster membership authority, and recovery control. It exposes narrow typed contracts. All other services are replaceable and interact with the kernel by commands, queries, and durable events.

6. Authoritative State Ownership

State domain	Authoritative owner	Derived / replaceable forms
Identity, delegation, authentication	Identity & Authority Service	Session caches, UI claims, presence assertions
Policy, exceptions, approvals	Policy System	Signed evaluation bundles, local evaluators
Experience and audit	Experience Ledger / Audit Service	Indexes, summaries, forensic projections
Knowledge and patterns	Knowledge Service	Embeddings, caches, search indexes
Intent, plan, transaction, outcome	Intent Manager / Planner / Transaction Coordinator	Dashboards, work queues, provider-specific task representations
Configuration and secrets references	Configuration Plane / Secrets Service	Node-local verified bundles, short-lived credentials
Node and provider observations	Node Registry / Provider Registry	Health views, benchmarks, scheduling projections

7. Request-to-Outcome Flow

- A Presence submits a structured request to HAL Core. The request carries identity, context, intent, priority, evidence, classification, and correlation metadata.
- Identity, policy, and trust checks establish whether the request can be understood, planned, or acted upon.
- The Intent and Planning services derive an approved plan. The Decision Engine records alternatives, evidence, uncertainty, and policy constraints.
- The Execution Orchestrator binds capability contracts to current providers and nodes, opens a transaction, reserves resources, and runs only within delegated authority.
- Providers return results, evidence, health, and execution facts. They never directly mutate authoritative state.
- Verification evaluates the result. The transaction completes, compensates, pauses, recovers, or escalates. Experience, audit, knowledge, and outcome records are updated through their authoritative owners.

8. Trust Boundaries

Boundary	Required control
Presence ↔ HAL Core	Continuous authentication, authorization, disclosure appropriate to Presence privacy and confidence.
Kernel ↔ governed service	Typed contract, least privilege, correlation, auditable policy decision, no direct database ownership violation.
Core ↔ node runtime	Mutual authentication, encrypted transport, signed configuration, health evidence, delegated execution envelope.
Provider ↔ adapter	Capability-scoped credential, provider fitness evidence, timeout/retry semantics, output verification.
HAL ↔ external trust domain	Treaty, Constitutional Firewall, provenance validation, data minimization, revocation and audit.

9. Operating Modes & Failure Posture

HAL exposes Normal, Degraded, Restricted, Safe Mode, Recovering, Maintenance, Emergency, and Offline modes. A failure may reduce availability or capability but cannot silently change policy, identity, authority, authentication requirements, trust thresholds, or constitutional rules. Critical state is preserved through authoritative ownership, append-only journals, durable transaction checkpoints, independent backups, and recovery-first boot.

10. Deployment Baseline

The initial deployment uses a Mac mini as Primary Constitutional Host and HAL Core control plane; GX10 systems run HAL Node Runtimes for governed model and specialist execution; a NAS holds a warm, encrypted, restore-tested recovery replica plus independent backup. This is a deployment topology, not a permanent identity requirement. HAL remains portable as a continuity of constitution, governed state, and authority.

11. Interfaces

Interface class	Primary semantic
Command	Requests one authoritative state change; caller receives

Interface class	Primary semantic
	acceptance, rejection, or required approval.
Query	Reads declared consistency class from an authoritative owner or controlled projection.
Event	Immutable fact with identity, schema, correlation, causation, authorization, provenance, and integrity metadata.
Capability invocation	Bounded request to fulfill a capability contract under a transaction and delegation envelope.
Treaty exchange	Cross-domain message governed by an active Treaty and the Constitutional Firewall.

12. Guarantees

- No external model, provider, node, service, or Presence is a constitutional authority by default.
- Every durable authoritative state domain has one owner, versioned mutation, append-only history, and auditable recovery path.
- Every meaningful action has a transaction, policy context, authorization context, evidence, and explainable outcome.
- All cross-process, device, or trust-boundary communication is authenticated, integrity-protected, and encrypted; sensitive exchanges are end-to-end protected.
- Degradation is disclosed. System rules do not silently change because a dependency fails.
- The architecture remains implementation-neutral: models, adapters, nodes, and non-kernel services are replaceable without changing HAL identity.

13. Constitutional Traceability Audit

Constitutional decisions	Chapter 1 implementation coverage
1–7	Core boundary, modular services, CQE contracts, event journal, state ownership, immutable identity, Self Model.
8–16	Engagement tiers, decision framework, capabilities, knowledge, planning, provider registry, graph separation, orchestration.
17–25	Nodes, resource constraints, lifecycle, work model, scheduling, recovery, communication, consistency, policy.
26–35	Trust, authentication, introspection, distributed execution, memory, intent, communication, learning, reasoning, transactions.
36–45	Capability stack, kernel boundary, supervision,

Constitutional decisions	Chapter 1 implementation coverage
	configuration, observability, resources, durability, updates, time, Presences.
46–58	Initiative, coordination, ownership, sovereignty, verification, self-description, environment, wisdom, attention, judgment, restraint, outcomes, evolution.

14. Internal Engineering Review

Check	Result
Constitutional conflicts	None identified. Book I remains the governing authority.
Authority ambiguity	Resolved through single-owner durable state and kernel-only constitutional mutation.
Failure posture	Explicit modes, safe degradation, recovery journals, and independent backup topology included.
Premature technology coupling	Avoided. Specific transports, databases, and frameworks are deferred to later chapters.
Owner review required	None. This chapter chooses implementation structure without altering constitutional philosophy or Owner authority.

15. Completion Status

Chapter 1 is complete as an architectural foundation. Subsequent chapters will refine the runtime, services, data models, interfaces, and deployment mechanisms named here without changing the authority model established in this chapter.

Chapter 2 — Runtime Model

Field	Value
Status	Architect draft complete; constitutional and engineering review complete
Version	0.1
Scope	Boot, lifecycle, supervision, operating modes, degradation, shutdown, and recovery
Constitutional basis	Decisions 17–25, 28–29, 35, 38–45, 47, 50–51, 54, and 56
Owner review	None required at this stage

1. Purpose

The runtime model defines how HAL exists while operating. It specifies the order in which HAL earns the right to become active, the state transitions it may enter, how it detects and contains failure, and how it recovers without changing its constitutional rules. A running process is not HAL merely because it starts; it becomes an active HAL runtime only after identity, policy, protected state, and health requirements have been verified.

2. Runtime Principles

- Bootstrap constitutional controls before optional intelligence or execution services.
- Treat every runtime state as explicit, observable, and auditable.
- Supervise services independently; isolate fault domains rather than letting one failure cascade.
- Permit capability loss and safe degradation, but never silent alteration of governance, authority, identity, trust, authentication, or policy.
- Prefer recovery, replay, and verification over unexamined restart.
- Make resource use, health, uncertainty, and operating limits part of the runtime self-model.

3. Runtime Constituents

Constituent	Runtime responsibility	Failure consequence
Bootstrap Manager	Builds the verified runtime in dependency order and selects the initial operating mode.	HAL remains unavailable or enters Safe Mode.
Constitutional Kernel	Establishes root identity, policy and authorization authority, protected audit, and commit coordination.	No constitutional mutation or protected action may proceed.
Service Supervisor	Starts, observes, isolates, restarts, and retires governed services.	Affected capability is degraded or suspended.
Resource Governor	Accounts for CPU, memory, GPU, storage, network, power, thermal,	Admission control or bounded degradation applies.

Constituent	Runtime responsibility	Failure consequence
	and attention budgets.	
Mode Manager	Maintains the authoritative operating mode and allowed transitions.	Defaults to the more restrictive safe state.
Health and Evidence Collector	Collects liveness, readiness, integrity, performance, and cognitive-health evidence.	Confidence falls; unverified components are restricted.
Recovery Manager	Coordinates journals, checkpoints, replay, restoration, and gradual rejoin.	Recovery holds the affected domain until verification completes.

4. Startup and Admission Sequence

Startup is a staged admission process. Each stage establishes evidence required by the next. A stage may halt or restrict the runtime; no stage may be skipped because a dependent service is unavailable.

Stage	Required evidence	Outcome
0. Hardware and host admission	Host identity, secure-time availability or bounded uncertainty, storage accessibility, minimum resource baseline.	Host is accepted as a candidate runtime host.
1. Kernel bootstrap	Root identity material, constitutional version, protected configuration, audit journal integrity.	Constitutional Kernel becomes available in restricted bootstrap state.
2. State admission	Ledger consistency, checkpoint validation, policy version verification, secrets reference availability.	Authoritative state is opened read-only until recovery checks pass.
3. Core-service admission	Identity, policy, audit, transaction, mode, supervisor, and observability readiness.	Core control plane becomes ready.
4. Capability admission	Node/provider identities, capability manifests, health, current delegation, treaty status.	Capabilities are registered but not necessarily schedulable.
5. Active admission	Readiness quorum, current Self Model, verified runtime mode, declared limitations.	HAL accepts normal or degraded work consistent with mode.

5. Runtime State Machine

The Mode Manager owns the current runtime mode. Every transition creates an immutable mode-transition event containing cause, evidence, affected capabilities, authority, and recovery criteria.

Mode	Permitted behavior	Exit criteria
Bootstrap	Identity and protected-state verification only. No external actions.	Kernel and state admission succeeds.
Normal	All healthy, authorized capabilities may operate under ordinary policy and transaction controls.	Detected degradation, scheduled maintenance, emergency, or shutdown.
Degraded	Healthy capabilities operate with disclosed limitations; scheduler excludes unfit components.	Required health and verification evidence restore confidence.
Restricted	Local reasoning, monitoring, evidence collection, and safe read operations; no protected permanent mutations.	Partition or integrity condition is reconciled and verified.
Safe Mode	Read-only inspection, recovery, audit, and Owner-authorized repair paths only.	Root cause is addressed and recovery verification passes.
Recovering	Journal replay, reconciliation, restore, integrity checks, and staged re-admission.	Authoritative state and readiness gates pass.
Maintenance	Approved scoped maintenance; unrelated work is isolated or deferred.	Maintenance change is verified, rolled back, or concluded.
Emergency	Priority routing for an active emergency under explicit emergency policy.	Emergency closes; post-incident review and reconciliation begin.
Offline	No external connectivity assumed. Local rules still apply; network-dependent capabilities are unavailable.	Connectivity and peer identity are re-established and verified.

6. Service Lifecycle and Supervision

Every governed service declares identity, dependency contracts, readiness criteria, resource limits, failure behavior, restart policy, state ownership, and evidence produced. The supervisor may restart a failed service, move work, rebuild a projection, or isolate a node. It may not modify the service's policy, delegation, constitutional role, or persistent authoritative data outside the established recovery path.

- A service transitions through Defined, Starting, Ready, Busy, Draining, Stopped, Failed, Quarantined, or Recovering.
- Liveness proves that a service is reachable; readiness proves it is fit to receive a declared workload. Neither substitutes for authorization or trust.
- Restart budgets prevent a crash loop from consuming resources or creating noisy, misleading health evidence.
- Stateful services drain or checkpoint before planned retirement. Stateless services may be replaced after contract compatibility and provenance checks.
- A quarantined service or node may collect evidence but receives no work that can change protected state until re-admitted.

7. Resource and Attention Admission

The Resource Governor evaluates physical and cognitive resource budgets before work begins. It uses the scheduling and attention systems to select the smallest adequate execution scope. Resource exhaustion is a source of evidence, not an excuse to bypass controls.

Budget	Examples of enforcement
Physical	CPU, GPU, memory, storage, thermal, power, and network headroom; admission limits and node fitness checks.
Cognitive	Reasoning, planning, retrieval, simulation, reflection, and learning allocations; attention budgets and starvation review.
Safety	Maximum concurrency for physical actions, transaction blast radius, rollback capacity, and verification capacity.
Privacy and treaty	Data classification, permitted destinations, retention, and disclosure bounds.
Time	Deadlines, time uncertainty, lease duration, retry horizon, and stale-evidence limits.

8. Failure Detection and Containment

Failure detection combines heartbeat, readiness, integrity, behavior, resource, and cross-subsystem evidence. The system avoids interpreting a missing heartbeat as a definitive explanation. It records a hypothesis, confidence, scope, and recommended containment action.

Failure class	Containment response
Single optional service	Isolate, restart within budget, route to a compatible alternative, and disclose degraded capability.
Stateful core service	Suspend affected mutations; preserve evidence; fail closed or safe according to the service declaration.
Node or provider integrity concern	Quarantine execution authority, collect evidence, require controlled recovery before rejoin.

Failure class	Containment response
Network partition	Enter Restricted mode for isolated partitions; prohibit canonical commits and governance actions until reconciliation.
Resource exhaustion	Apply admission control, defer low-priority work, reduce optional workloads, and preserve critical recovery capacity.
Constitutional integrity concern	Enter Safe Mode; stop protected mutations; escalate to Owner Authorization Ceremony for any repair requiring protected change.

9. Shutdown, Handoff, and Recovery

A planned shutdown is a transaction: HAL drains active work, records handoff state, closes or compensates transactions, checkpoints durable state, preserves audit and mode evidence, and verifies that recovery material is available. An unplanned shutdown is treated as an incident. On restart, HAL does not assume previous work succeeded; it reconstructs state from durable facts and verification.

- Graceful shutdown stops new work first, then drains reversible work, then handles protected transactions according to their recovery contract.
- Every in-flight transaction has a durable status: pending, committed, compensated, abandoned with evidence, or awaiting Owner decision.
- Recovery replays authoritative journals, verifies checkpoints, identifies uncertain effects, and uses reconciliation rather than duplicate execution.
- Nodes and services rejoin gradually: identity, integrity, policy, trust, state synchronization, health, and capability fitness are separately verified.
- Restoration is not complete until a recovery verification record proves the restored system can read, reason, enforce policy, and execute only within its permitted mode.

10. Runtime Observability and Self Model

The runtime publishes evidence to the operational and cognitive self-model. It reports current mode, active limitations, health dimensions, resource headroom, dependency status, admission decisions, restart history, recovery progress, time confidence, and calibration concerns. A status response is evidence-backed; a language model may render it, but it does not invent it.

11. Runtime Guarantees

- HAL MUST establish constitutional identity and protected state before accepting protected work.
- HAL MUST record mode changes, recovery transitions, and authoritative state changes as durable evidence.
- HAL MUST NOT treat local network location, a successful heartbeat, or service availability as sufficient trust or authorization.
- HAL MUST NOT silently change its rules as a consequence of degradation, partition, overload, recovery, or emergency.
- HAL MUST apply the declared failure behavior of each capability and make reduced capability visible to affected participants.
- HAL MUST preserve an auditable path from request, through runtime mode and resource admission, to execution evidence and outcome.

12. Constitutional Traceability Audit

Constitutional decisions	Chapter 2 implementation coverage
17–25	Logical execution nodes, resource limits, lifecycle, work scheduling, capability allocation, recovery, secure communication, consistency, policy enforcement.
28–29	Operational/cognitive self-awareness, health forecasting inputs, fault tolerance, split-brain restrictions, self-healing boundaries, failure declaration.
35, 38–45	Transactions, supervision, configuration integrity, observability, resources, durable recovery, safe lifecycle change, temporal confidence, Presence continuity.
47, 50–51	Distributed coordination, reality modes, verification gates, self-description, identity continuity, constitutional mirror inputs.
54, 56	Attention budgets, starvation prevention inputs, explicit limits, uncertainty-aware restriction and escalation.

13. Internal Engineering Review

Check	Result
Constitutional conflicts	None identified. Runtime behavior preserves Book I authority boundaries.
Lifecycle completeness	Startup, admission, active operation, degradation, containment, shutdown, and recovery are covered.
Failure posture	Mode transitions, service isolation, resource admission, and partition restrictions are explicit.
Unnecessary coupling	Specific process managers, databases, and orchestration frameworks are intentionally deferred.
Owner review required	None. The chapter defines implementation mechanics without changing constitutional philosophy, values, or authority.

14. Completion Status

Chapter 2 is complete. It establishes the runtime contract that later service, event, storage, and deployment chapters must satisfy. Chapter 3 will define the Constitutional Kernel and the narrow enforcement interfaces on which the runtime depends.

Chapter 3 — Constitutional Kernel

Field	Value
Status	Architect draft complete; constitutional and engineering review complete
Version	0.1
Scope	Protected enforcement boundary, root state, kernel contracts, mutation protocol, and recovery authority
Constitutional basis	Decisions 1–7, 24–29, 35, 38–43, 47–51, 56, and 58
Owner review	None required at this stage

1. Purpose

The Constitutional Kernel is the smallest HAL subsystem that must remain trustworthy for HAL to preserve constitutional identity. It is not a general-purpose application framework, planner, model host, or database. It holds the authority to establish root identity, evaluate protected policy, validate Owner authorization, preserve protected audit evidence, coordinate irreversible commits, and control recovery admission. Every other service is replaceable; the kernel is deliberately narrow, explicit, and continuously verifiable.

2. Design Boundary

The kernel enforces constitutional constraints. Governed services supply reasoning, proposals, evidence, state transformations, and execution results. The kernel decides only whether a protected request has the identity, authority, policy context, evidence, verification, and durable record required to proceed. It does not substitute its own judgment for the Intent, Trust, Planning, or Judgment services.

Kernel responsibility	Explicit non-responsibility
Root identity and trust anchor	Natural-language interpretation or personality
Protected policy and authorization evaluation	Planning, ranking, or goal selection
Owner Authorization Ceremony validation	Model inference or provider routing
Constitutional version and invariant admission	Search, retrieval, or knowledge graph maintenance
Protected audit append and evidence binding	Direct control of optional capabilities
Commit coordination for protected state	Application-specific business logic
Recovery and re-admission authority	Silent repair of constitutional state

3. Kernel Root State

Kernel root state is compact, versioned, cryptographically protected, and independently recoverable. It is sufficient to determine which constitutional identity is being restored and whether a proposed protected action is permitted. It never depends on a single model, provider, hardware host, or presentation layer.

Root-state record	Purpose
Constitutional Identity Record	Stable HAL identity, root public keys, approved recovery roots, and continuity metadata.
Owner Authority Record	Exactly one active constitutional Owner identity, protected succession state, and owner-authentication requirements.
Constitution and Invariant Registry	Signed constitutional versions, immutable invariant declarations, effective dates, compatibility markers, and amendment lineage.
Protected Policy Registry	Active constitutional and protected policies, policy version lineage, and policy-evaluator manifests.
Authorization Ceremony Registry	Pending and completed protected authorizations, exact change bindings, expiry, replay protection, and audit references.
Protected Audit Anchor	Append-only audit chain roots, checkpoints, witness references, and integrity-verification history.
Recovery Admission Record	Trusted recovery material, last verified restore state, emergency restrictions, and re-admission evidence.

4. Kernel Contracts

Kernel interfaces are typed, versioned contracts. A caller submits a command or query under an authenticated service identity. The kernel returns a signed decision, a narrowly scoped capability token, a denial with rationale, or a request for additional evidence or Owner authorization.

Contract	Primary input	Kernel result
EvaluateProtectedAction	Actor, intent, scope, policy context, evidence, requested effect.	Allow, deny, require verification, require Owner ceremony, or route to recovery/safe mode.
BeginProtectedTransaction	Authorized plan, rollback/compensation path, blast radius, verification requirement.	Transaction lease and immutable policy/authority binding.
CommitProtectedMutation	Transaction result, verification evidence, state version precondition, audit payload.	Atomic commit, compensating action, or durable pending/recovery state.

Contract	Primary input	Kernel result
ValidateOwnerAuthorization	Exact change identifier, current owner session evidence, fresh authorization factor.	One-time signed authorization or denial.
VerifyConstitutionalState	Constitution version, invariant set, root-state evidence, audit anchors.	Admit, restrict, or enter Safe Mode.
AdmitRecovery	Restore evidence, integrity report, reconciliation plan, current policy and identity state.	Recovery admission plan or rejection.
IssueDelegatedExecutionEnvelope	Authorized actor, capability scope, time limit, transaction/correlation context.	Non-transferable, auditable, narrowly scoped execution envelope.

5. Protected Mutation Protocol

A protected mutation is any state change that affects constitutional rules, protected policy, Owner authority, identity, trust requirements, authentication requirements, audits, quarantines, Treaties, capability classes, or protected permanent state. Protected mutations use a deliberate prepare–verify–authorize–commit sequence.

- **Prepare:** the requesting service creates a complete change object with exact intended effect, alternatives, affected state, risk, rollback or recovery plan, and required verification.
- **Validate:** the kernel checks identity, delegation, policy, constitutional version compatibility, transaction preconditions, current operating mode, and evidence freshness.
- **Verify:** the Verification Service supplies required simulation, shadow, canary, integrity, or recovery evidence; the kernel records its result without inventing confidence.
- **Authorize:** if the change crosses a protected threshold, the kernel requires the Owner Authorization Ceremony bound to the exact immutable change identifier.
- **Commit:** the kernel writes the protected audit record and commits only if state versions, authorization, verification, and policy bindings still match.
- **Observe:** outcome evidence, rollback readiness, and post-change health are attached. A changed proposal invalidates prior authorization and must restart the ceremony.

6. Owner Authorization Ceremony Enforcement

The kernel enforces the constitutional distinction between deliberative assent and verified authorization. Ordinary conversational agreement, including “yes,” “lock it in,” or “go ahead,” is never an execution credential for a protected change. The kernel accepts only a fresh, time-limited, non-replayable authorization bound to one exact change object and supported by current Owner identity and liveness evidence.

Requirement	Kernel enforcement
Exact binding	Authorization includes the immutable change identifier, affected records, and canonical change digest.
Freshness	Authorization expires quickly and cannot be reused after completion, timeout, or material proposal change.
Identity and liveness	The kernel verifies active Owner authentication

Requirement	Kernel enforcement
	evidence; a successful historical session is insufficient.
Factor independence	Authorization factor is separate from ordinary conversation and may combine passkey, hardware key, PIN, biometrics, signed Owner device, or future approved liveness method.
Auditability	The ceremony records the proposal, disclosures, authentication evidence class, authorization result, and execution outcome without unnecessarily exposing secrets.
Failure behavior	If any required check is unavailable or uncertain, the change remains pending or is denied; standards never relax because a factor is unavailable.

7. Invariants and Safe Mode

The kernel treats an invariant conflict, untrusted root state, broken audit chain, unresolved Owner identity conflict, or unauthorized constitutional mutation as a constitutional incident. It enters Safe Mode for affected protected domains, preserves evidence, and permits only inspection, recovery, and Owner-authorized repair paths.

- There is exactly one active constitutional Owner identity at any time.
- No protected state change is valid without a corresponding immutable audit record and applicable authorization evidence.
- No service may impersonate another service or broaden its delegated authority.
- No node, provider, presence, network location, or historical trust score substitutes for current identity, authorization, and policy evaluation.
- Failures may change capabilities but may never silently change HAL's rules.
- Constitutional identity persists through implementation replacement only when root state, constitutional lineage, governed memory, audit continuity, and authority are preserved.

8. Kernel Availability and Recovery

The kernel is replicated or recoverable according to the deployment's protected-state durability policy, but it is logically singular: concurrent partitions cannot independently make protected canonical commits. A partition lacking required quorum or current constitutional state enters Restricted mode. Recovery uses signed constitutional mirrors, protected audit checkpoints, independent backups, and reconciliation evidence before resuming protected work.

Condition	Kernel behavior
Optional governed service unavailable	Kernel continues enforcing protected controls; related capability is unavailable or degraded.
Kernel replica disagreement	Reject protected commits; preserve evidence; reconcile using audit anchors and authorized recovery procedure.
Audit anchor mismatch	Enter Safe Mode for protected mutations; verify lineage and restore or reconcile before admission.

Condition	Kernel behavior
Owner identity uncertainty	Deny protected Owner actions; allow non-protected, separately authorized operation where policy permits.
Constitution version mismatch	Admit only the verified compatible version; restrict affected services and require governed amendment/recovery path.
Host replacement or migration	Re-establish root state, verify recovery material, re-bind host identity, and perform staged runtime admission.

9. Security and Privacy Posture

Kernel credentials and root records use hardware-backed or equivalently protected key material where available. Secrets are referenced rather than exposed to ordinary services. The kernel minimizes retained sensitive payloads: it records evidence identifiers, digests, classifications, and access decisions unless protected content is essential to constitutional recovery or audit. All kernel contracts require authenticated, integrity-protected communication; sensitive payloads remain encrypted to authorized recipients.

10. Kernel Observability

The kernel publishes a bounded Self Model projection: constitutional version, invariant verification state, current mode, protected-policy version, audit-anchor health, pending authorization ceremonies, recovery status, and declared limitations. It does not expose secret material, raw authorization factors, or unnecessary private payloads. Any inconsistency between the projection and protected root state is a constitutional incident.

11. Guarantees

- The kernel **MUST** be the only path for constitutional and other protected commits.
- The kernel **MUST** bind protected decisions to identity, authority, policy, evidence, verification, time, and immutable audit evidence.
- The kernel **MUST NOT** infer protected authorization from conversational language, a prior approval, a cached session, or a network location.
- The kernel **MUST** fail safe for uncertain protected state and **MUST** preserve the evidence necessary for recovery and Owner review.
- The kernel **MUST** expose narrow contracts so governed services remain replaceable without gaining constitutional authority.
- The kernel **MUST** verify continuity before admitting restored, migrated, or partitioned protected state.

12. Constitutional Traceability Audit

Constitutional decisions	Chapter 3 implementation coverage
1–7	Constitutional core boundary, event/audit journal, identity continuity, self-model evidence, governed service contracts.
24–29	Policy enforcement, evidence/trust separation, continuous authentication, introspection, distributed restrictions and recovery.

Constitutional decisions	Chapter 3 implementation coverage
35, 38–43	Protected transactions, supervision boundary, configuration integrity, observability, durability, recovery, safe change.
47–51	Coordination without authority transfer, ownership and succession, sovereignty, verification, self-description and constitutional continuity.
56, 58	Uncertainty-driven restraint, explicit Safe Mode, amendment lifecycle support, and protected invariants.

13. Internal Engineering Review

Check	Result
Constitutional conflicts	None identified. The kernel implements, rather than extends, Book I authority.
Scope discipline	Planning, learning, model execution, and external integration remain outside the kernel.
Authorization safety	Exact-change binding, freshness, liveness, independent factor, and audit requirements are explicit.
Recovery posture	Partition, mismatch, migration, and audit-integrity failure paths enter restriction or Safe Mode.
Owner review required	None. Kernel boundaries and contracts are implementation choices consistent with already locked constitutional requirements.

14. Completion Status

Chapter 3 is complete. It establishes the narrow enforcement boundary that permits later policy, identity, audit, transaction, and recovery services to evolve without diluting constitutional authority. Chapter 4 will specify Identity and Continuity.